

AI Newsletter Generation & Distribution Platform

Complete technical handover — architecture, module reference,
swap guide and operations runbook

Python 3.11–3.14

Streamlit

Groq · gpt-oss-120b

SQLite + Alembic

Zero local inference

Document Handover Pack v1.0 — single source of truth

Covers Milestones M0–M10 (manual pipeline + autonomous agent)

Scale 94 source files · ~16,900 lines · 1,168 tests · 93% coverage

Owner Sidhant — sidhant@deephorizon.io

Date 18 August 2026

§ Contents

- 1** What this system does
The problem, the pipeline, the two ways to run it
- 2** Quick start on a fresh machine
Ten minutes from clone to a working dashboard
- 3** Architecture
Layers, the import rule, the two processes
- 4** How the system flows
Six diagrams: manual, automated, approval, guards, states
- 5** Timing — exactly what is calculated when
Discovery cadence, the 3rd-Wednesday rule, the send gate
- 6** Module reference
Every package, module and public function
- 7** Data model
Twelve tables and what each is for
- 8** Configuration
The two layers, every variable, the secret rule
- 9** Swapping things out
LLM, email, template, blog source, database, host, auth
- 10** Operations runbook
Start, stop, monitor, and the failures you will actually hit
- 11** Testing & quality gates
Six gates, how to run them, what each protects
- 12** Security
Secrets, tokens, sessions, and the send guards
- 13** Decision register
What was chosen, what was reversed, and why
- 14** Known gaps & next steps
What is genuinely not done yet

1 What this system does

Marketing at Vays used to read partner blog posts by hand, summarise them, rewrite them into customer-facing copy, format the HTML, and send the result. This platform does that end to end — and can now do it unattended, once a month, with a human approving the send.

1.1 The pipeline

Everything the platform does is one pipeline. The manual and automated modes run *the same* pipeline; they differ only in who starts it and who clicks send.

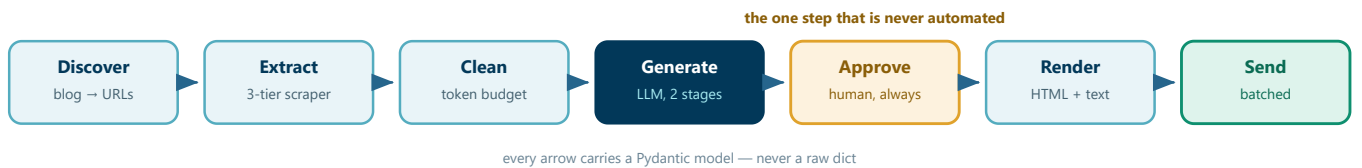


Figure 1 — The pipeline. Seven stages. Human approval sits between generation and sending by design (D-5); it is the primary mitigation for the LLM inventing a fact about an OEM partner, and it is not removable by configuration.

1.2 Two ways to run it

MANUAL — THE DASHBOARD

A person pastes blog URLs into **Generate Newsletter**, reviews and edits the generated copy on **Campaign Preview**, then sends. Full control, used for one-off or special campaigns. This is milestones M1–M9.

AUTOMATED — THE AGENT

A background worker checks the Vays blog on a schedule, generates a newsletter from any new post, emails management a secure approval link, and sends to the whole list on the 3rd Wednesday of the month. This is M10.

Both write to the same database and appear in the same **Campaign History**. The automated mode is simply the manual mode with a scheduler and an email-based approval step in front of it.

1.3 The three constraints that shaped everything

CONSTRAINT	WHAT IT MEANS IN THE CODE
Open-source LLM only	The <i>model</i> must be open-weight; the <i>host</i> need not be. Groq serves only open-source models, so <code>openai/gpt-oss-120b</code> (Apache 2.0) satisfies this more cleanly than self-hosting would. See D-2.
Zero local inference	No model, weights, or inference runtime ever runs on the developer machine. Machine-enforced: <code>scripts/check_no_local_inference.py</code> fails the build if <code>torch</code> , <code>transformers</code> , <code>ollama</code> , <code>vllm</code> or friends appear in any requirements file or import. It runs <i>first</i> in CI.
Handover-ready	Another developer must clone, configure and run this on a different Windows PC from the documentation alone. That is what this document is for.

2 Quick start on a fresh machine

Target: a working dashboard in about ten minutes, assuming Python is installed.

2.1 Prerequisites

Python	3.11 – 3.14 (developed and verified on 3.14.3 , Windows 11 AMD64)
OS	Windows 10/11 primary. The code is pathlib -based and CRLF-safe; Linux works and is covered by CI, but run.bat is Windows-only.
RAM	Any. All inference is remote — this runs on an 8 GB laptop with no GPU.
Network	Outbound HTTPS to api.groq.com , the blog, and your mail provider.
Not needed	Node.js, Docker, Redis, a GPU, or any paid service beyond a mail provider.

2.2 The six commands

```
# 1 – get the code and a virtual environment
git clone <repo-url> vays-newsletter-ai
cd vays-newsletter-ai
py -3.12 -m venv .venv
.venv\Scripts\activate

# 2 – install exactly the versions that were tested
pip install -r requirements.lock.txt

# 3 – configuration
copy .env.example .env
python -c "import secrets; print(secrets.token_urlsafe(48))"
#   paste that into APP_SECRET_KEY= in .env, then add GROQ_API_KEY=

# 4 – create the database (12 tables, 5 migrations)
alembic upgrade head

# 5 – create the first login. No default password exists anywhere.
python -m scripts.create_user

# 6 – run
run.bat           # → http://localhost:8501
```

USE THE LOCKFILE

requirements.lock.txt pins all 132 packages for a byte-identical install. **requirements.txt** holds the loose ranges and is what you edit when adding a dependency; regenerate the lock with **pip freeze** afterwards.

2.3 What **run.bat** checks before it starts

A fresh machine fails in four predictable ways, so the launcher checks for each and names the fix rather than showing a stack trace:

CHECK	WHY IT EXISTS
Virtual environment exists	Otherwise streamlit resolves to a global install or nothing.

CHECK	WHY IT EXISTS
<code>.env</code> exists	Startup would fail on a <code>ConfigurationError</code> with no clue that a file is simply missing.
Migrations applied	A schema behind <code>head</code> produces confusing <code>OperationalErrors</code> at first click, not at boot.
At least one user account	A login screen with no accounts is a dead end. This check is the difference between a two-minute start and an hour lost.

2.4 Running without the agent

`run.bat` starts **only the dashboard**. The automation is a separate process and is off by default (`AGENT_ENABLED=false`). Section 10 covers starting it. This separation is deliberate — see §3.3.

3 Architecture

3.1 The layers

A modular monolith: one process, four layers, and a hard rule about which way imports may point. The rule is not a convention — it is checked by `import-linter` as one of the six gates, so a violation fails the build.

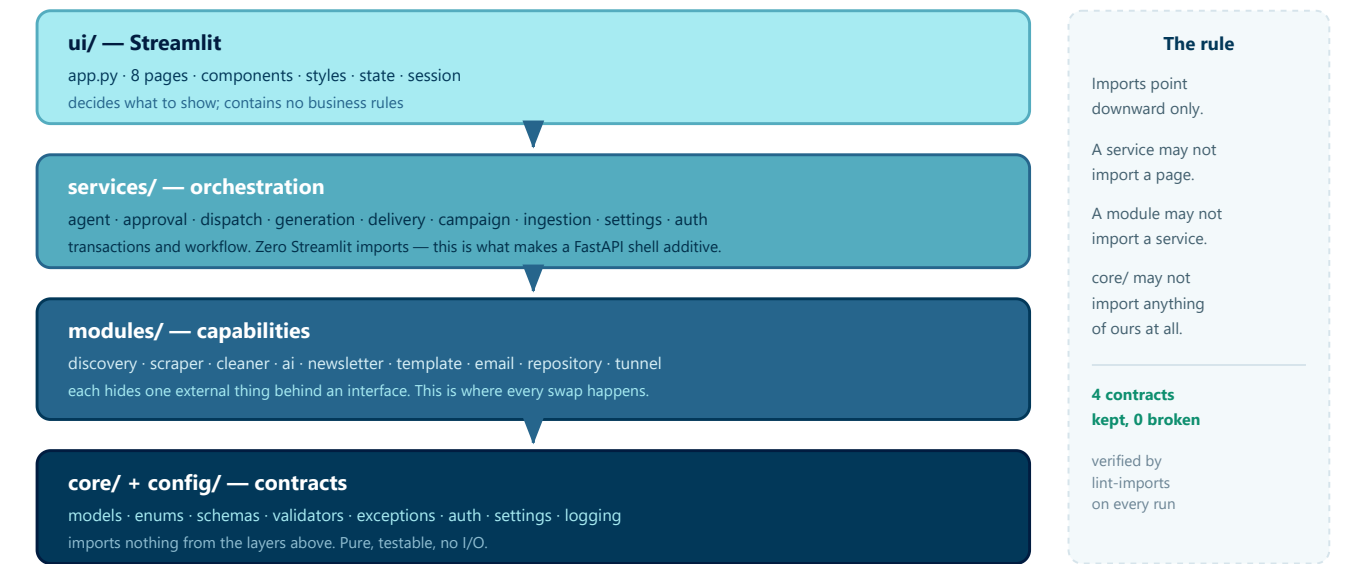


Figure 2 — Layers and the import rule. The value of the rule is concrete: because `services/` has no Streamlit import, the entire business layer is callable from a script, a test, a scheduler, or a future HTTP API. `agent_worker.py` is the proof — it drives the whole pipeline with no UI present.

3.2 What lives where

PATH	FILES	CONTAINS
<code>core/</code>	7	Pydantic models, enums, JSON schemas, validators, exception hierarchy, password & token crypto
<code>config/</code>	3	Settings sections, structlog setup, path constants
<code>modules/</code>	36	The nine capability packages — every external dependency is behind one of them
<code>services/</code>	17	Orchestration; one service per user-visible workflow
<code>ui/</code>	14	Eight pages plus shared components, styles, session and state helpers
<code>prompts/</code>	4 + shared	Versioned YAML prompts with Jinja2 bodies
<code>templates/</code>	3 + 1	Three email templates; internal (approval) templates kept separate — see §9.3
<code>migrations/</code>	5	Alembic revisions
<code>tests/</code>	—	1,168 tests: unit, integration, e2e (<code>AppTest</code>)
<code>scripts/</code>	—	<code>create_user</code> , <code>check_no_local_inference</code> , <code>validate_prompts</code> , doc-link checker

3.3 Two processes, not one

This is the single most important operational fact in the document.

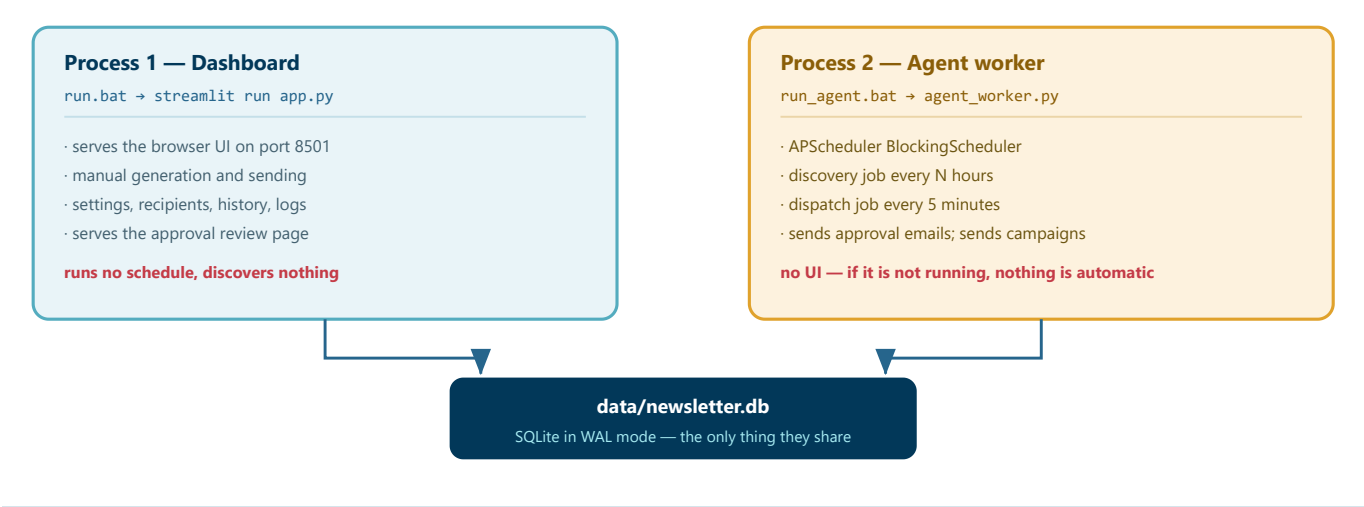


Figure 3 — The two processes. They never call each other; they coordinate purely through database rows. WAL mode is what allows the worker to write while the dashboard reads. **The most common support question — "why isn't the agent doing anything?" — is almost always that Process 2 was never started.**

WHY SEPARATE

Streamlit re-executes `app.py` top to bottom on every click, and a Streamlit session ends when the browser tab closes. A scheduler living inside that model would be restarted constantly, would run once per open tab, and would stop when the last tab closed. A separate process gets one clock, one instance, and a lifetime independent of anyone's browser.

3.4 External dependencies at runtime

SERVICE	REQUIRED?	USED FOR · FAILURE BEHAVIOUR
Groq API	required	All generation. Failures raise typed errors (<code>LLMRateLimitedError</code> , <code>LLMTimeoutError</code>); a circuit breaker opens after repeated failures so a dead endpoint fails fast instead of hanging every request.
Mail provider	required to send	SMTP (Gmail) or Brevo. <code>console</code> writes <code>.eml</code> files to <code>data/outbox/</code> and sends nothing — the safe default.
vaysinfotech.com	agent only	Blog discovery. Unreachable → the discovery job logs and returns; the next run retries.
ngrok	optional	Only to make approval links clickable from outside the LAN. Absent → links fall back to <code>http://localhost:8501</code> , which works fine when approving from the same machine.

4 How the system flows

4.1 The manual flow

What a marketing user does today, and the code path behind each click.

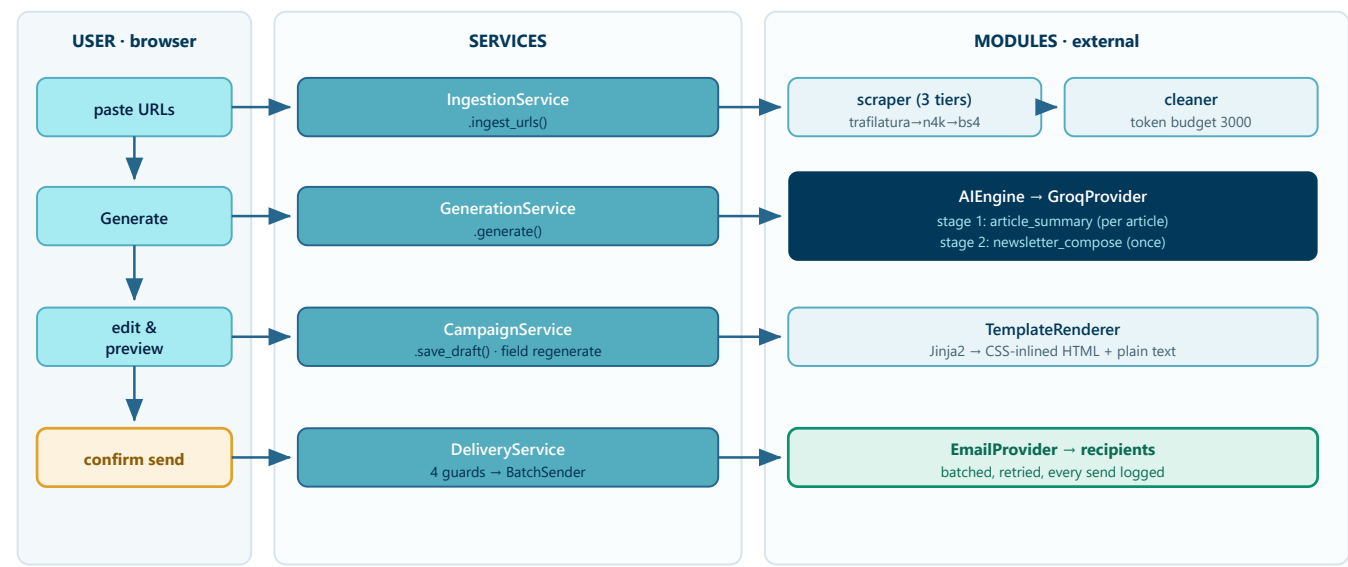


Figure 4 — Manual flow. Each row is one user action. The user never touches a module directly; the page calls a service, and only the service knows which modules are involved. That is why swapping the email provider (§9.2) changes no page code.

4.2 The automated agent flow

The same pipeline with a scheduler in front and an email approval in the middle. Two jobs run on different clocks, and they are deliberately independent.

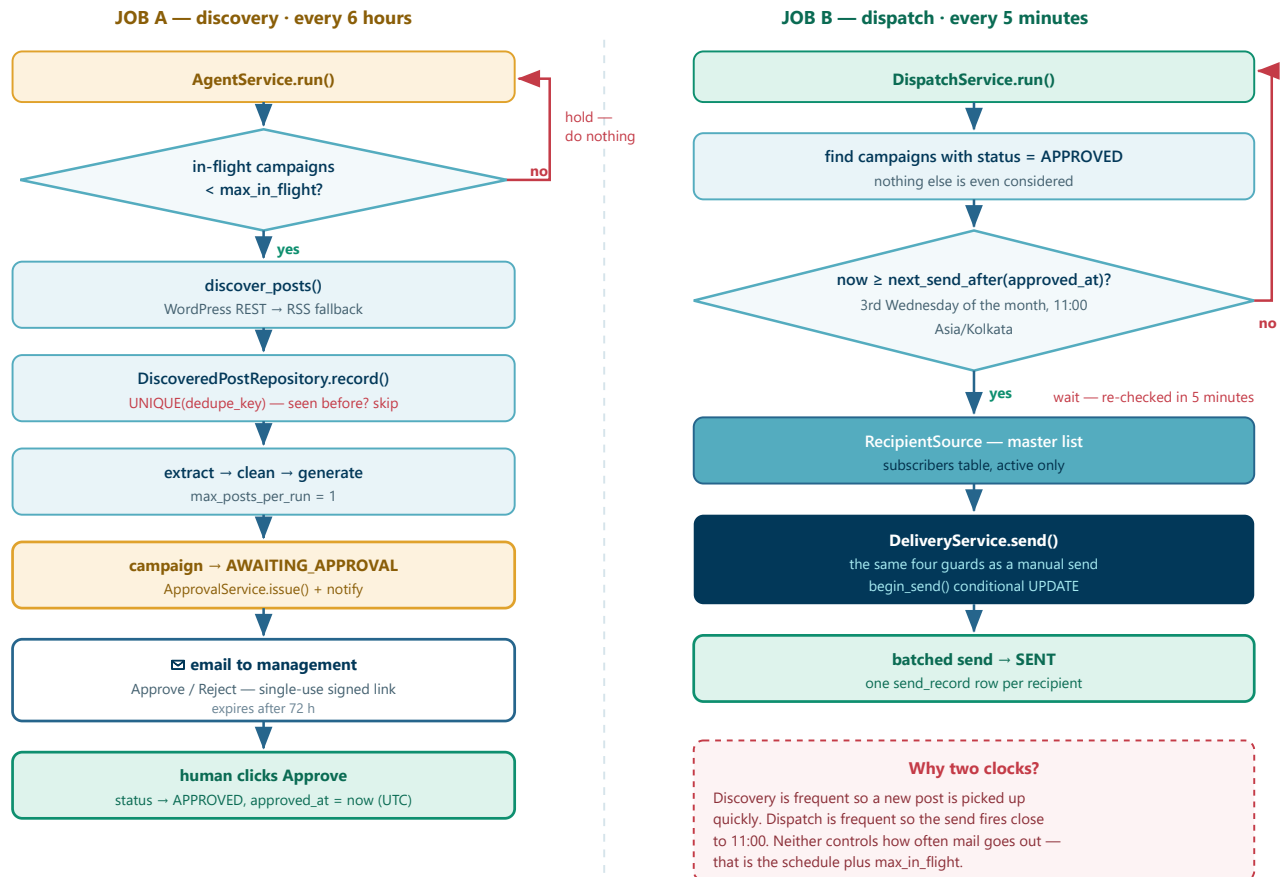


Figure 5 — Automated flow. The two jobs never call each other. Job A produces an `APPROVED` row (via a human); Job B consumes one. The `max_in_flight` check at the top of Job A is what enforces "one newsletter per month" — see §5.3 for why a per-run cap was not enough.

4.3 The approval link

The approval email contains a URL that must be safe to send through an inbox — it is, in effect, a password sitting in someone's mailbox. Five properties make that acceptable.

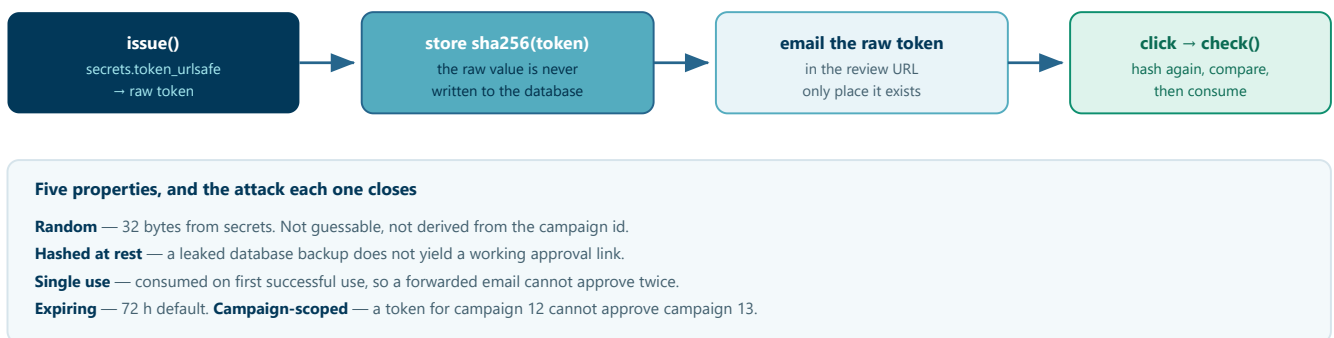


Figure 6 — Approval token lifecycle. Implemented in `services/approval_service.py`. Note the UTC discipline: SQLite returns naive datetimes, so `_as_utc()` normalises before every comparison — without it, every expiry check raised `TypeError`. That was a real bug, caught by a test.

4.4 The four guards between "send" and a customer's inbox

Every send — manual or automated — passes through all four. Each has a test named after the failure it prevents, because each corresponds to an email a customer would otherwise have received twice or should not have received at all.

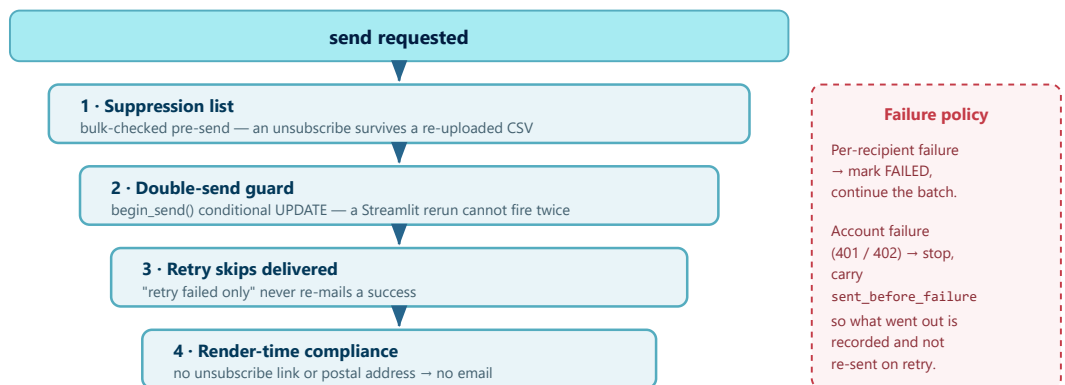
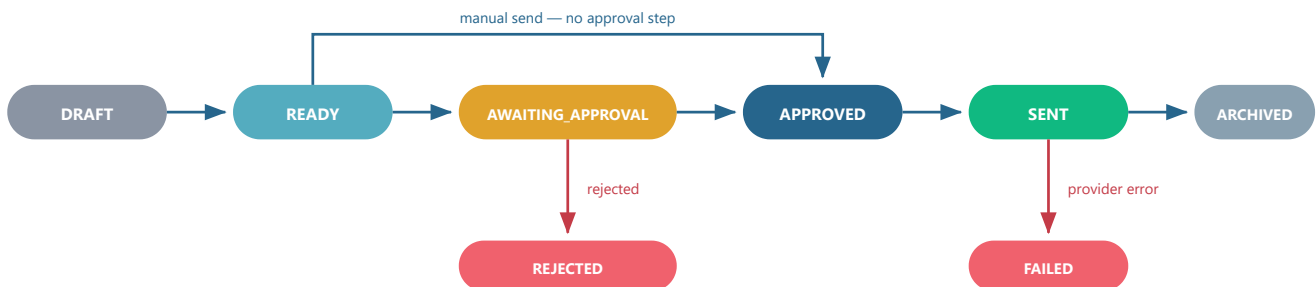


Figure 7 — The send guards. Guard 2 deserves emphasis: it is a conditional `UPDATE ... WHERE status IN (SENDABLE_STATES)`, so the database decides, not the application. `AWAITING_APPROVAL` is deliberately absent from `SENDABLE_STATES` — that single omission is what makes an unapproved campaign unsendable, through the guard that already existed.

4.5 Campaign states



`SENDABLE_STATES = (DRAFT, READY, APPROVED)`. Transitions are validated by `core.enums.can_transition()` — an illegal move raises rather than corrupting a row.

Figure 8 — Campaign state machine. The automated path adds `AWAITING_APPROVAL → APPROVED`; the manual path skips it. Both converge on `SENT` through the same code.

5 Timing — exactly what is calculated when

Three independent clocks, and they are frequently confused. Discovery cadence is not send frequency; the dispatch poll is not the send time. Only the schedule plus `max_in_flight` decides how often a customer receives mail.

5.1 The three clocks

CLOCK	DEFAULT	SETTING	WHAT IT ACTUALLY CONTROLS
Discovery	every 6 h	<code>AGENT_DISCOVERY_INTERVAL_HOURS</code>	How soon after publication a new blog post is <i>noticed</i> . Raising it does not send more email.
Dispatch poll	every 5 min	fixed in <code>agent_worker.py</code>	How precisely the send lands on 11:00. A 5-minute poll means a send fires by 11:05 at the latest.
Send schedule	3rd Wed, 11:00	<code>AGENT_SEND_*</code> (4 vars)	The only clock that decides when customers get mail.

5.2 How the send moment is computed

`Settings.next_send_after(approved_at)` in `config/settings.py` answers one question: given that a human approved at time T , what is the earliest legal moment to send? For the monthly schedule:

1. Take the month of T in `AGENT_TIMEZONE` (default `Asia/Kolkata`).
2. Find the **3rd Wednesday** of that month via `_nth_weekday_of_month()` — walk to the first Wednesday, add 14 days.
3. Set the clock to `AGENT_SEND_TIME` (`11:00`).
4. If that instant is already past, roll forward with `_add_months(+1)` and repeat.

The dispatch job then simply asks `now >= next_send_after(approved_at)`. Approving early is therefore always safe: approval never sends, it only makes a campaign *eligible*.

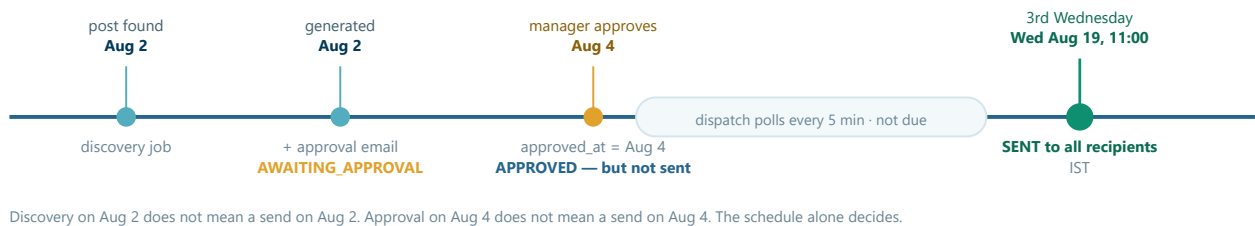


Figure 9 — A real month. Seventeen days pass between generation and delivery, and that is the intended behaviour, not a stall.

5.3 Why `max_in_flight` exists

A BUG WORTH UNDERSTANDING BEFORE YOU CHANGE ANY TIMING SETTING

"One newsletter per month" was first implemented as `max_posts_per_run = 1`. That caps each *run*, not the month — and with discovery running every six hours, a backlog of blog posts produced a queue of campaigns that all became due on the same 3rd Wednesday.

THIS HAPPENED: NINE NEWSLETTERS WENT OUT WITHIN TWO MINUTES ON 14 AUGUST.

The fix is `max_in_flight = 1`: the agent refuses to start any new work while a campaign is still unsent. Frequency is now bounded by the pipeline, not by the discovery cadence — which means discovery can safely run as often as you like.

5.4 Changing the schedule

```
# .env - or Settings → Agent in the dashboard, which needs no restart
AGENT_SEND_SCHEDULE=monthly # monthly | daily
AGENT_SEND_WEEKDAY=wednesday
AGENT_SEND_WEEK_OF_MONTH=3 # 1-5; ignored when schedule=daily
AGENT_SEND_TIME=11:00 # 24-hour HH:MM, anchored regex - "9:00" is rejected
AGENT_TIMEZONE=Asia/Kolkata
```

The dashboard renders the result in plain English via `describe_schedule()` ("3rd Wednesday of each month at 11:00 IST"), so a mis-set field is visible before it matters. `AGENT_SEND_TIME` is validated by an anchored regex rather than `time.fromisoformat`, which accepts `24:00` and silently means midnight.

6 Module reference

Every package, what it is responsible for, and its public surface. Private helpers (leading underscore) are omitted. Signatures are extracted from the source, so they are accurate as of this build.

6.1 `core/` — contracts and pure logic

Imports nothing from the rest of the application. No I/O, no network, no database. This is what makes it trivially testable and safe to import from anywhere.

`core/models.py`

Every object that crosses a layer boundary. 14 Pydantic models with `extra="forbid"`, so an unexpected field is an error rather than silent data loss.

MODEL	CARRIES
<code>ExtractedArticle</code>	Raw scraper output — text, title, tier used, word count
<code>CleanedArticle</code>	Normalised, deduped, truncated to the token budget
<code>Article</code>	A persisted article with its identity and provenance
<code>IngestionResult</code>	Per-URL success/failure for the whole ingest batch
<code>DiscoveredPost</code>	A blog post the agent found — URL, external id, published date
<code>ArticleSummary</code>	Stage-1 LLM output: key points, technical facts, relevance score
<code>NewsletterContent</code>	Stage-2 LLM output: subject, preview text, body blocks, CTA
<code>NewsletterDraft</code>	Generated content merged with user edits — what gets rendered
<code>GenerationRequest</code> / <code>GenerationParams</code> / <code>GenerationOptions</code>	Tone, length, audience, temperature — the inputs to a generation
<code>LLMResponse</code> , <code>Message</code>	Provider-neutral request/response shapes
<code>HealthStatus</code>	One probe result — healthy, latency, detail

`core/enums.py`

<code>CampaignStatus</code>	DRAFT · READY · AWAITING_APPROVAL · APPROVED · REJECTED · SENDING · SENT · FAILED · ARCHIVED
<code>PostState</code>	DISCOVERED · EXTRACTED · GENERATED · FAILED · SKIPPED — the agent's per-post progress
<code>SendStatus</code> , <code>SuppressionReason</code>	Per-recipient delivery outcome; why an address is suppressed
<code>Tone</code> , <code>LengthPreset</code> , <code>Audience</code> , <code>Category</code>	Generation controls exposed in the UI
<code>UserRole</code>	ADMIN · EDITOR — settings editing is admin-only

ExtractorTier	Which of the three extractors succeeded — recorded for the 88% measurement
<code>can_transition(current, target) -> bool</code>	The state machine. Rejects illegal moves.
<code>allowed_transitions(current) -> frozenset</code>	Used by the UI to decide which buttons to show

core/auth.py

All password and token cryptography. ~160 lines, deliberately auditable — D-15 refused a third-party auth package precisely so this file could be read in full at handover.

<code>hash_password(password) -> str</code>	bcrypt with a per-password salt
<code>verify_password(password, hashed) -> bool</code>	Constant-time comparison
<code>validate_password_strength(password) -> None</code>	Raises with a specific reason; used by <code>create_user</code> and the change-password form
<code>generate_password() -> str</code>	For admin-created accounts, so no default password ever exists
<code>sign_session_token(username, role, secret) -> str</code>	HMAC-signed, carries an expiry
<code>verify_session_token(token, secret) -> dict None</code>	None for forged, tampered or expired — the caller cannot accidentally distinguish them

core/validators.py · core/schemas.py · core/exceptions.py

<code>validate_url(url) -> str</code>	Normalises and applies the SSRF guard — see below
<code>is_public_address(host) · resolve_public_addresses(host)</code>	Refuses loopback, link-local, and RFC-1918 ranges. Without this, a pasted URL of http://169.254.169.254/ would make the server fetch its own cloud metadata.
<code>validate_email_address(value) -> str</code>	Used on CSV import and manual entry
<code>to_strict_schema(model) -> dict</code>	Pydantic → Groq strict JSON schema. Marks every property required and strips <code>minLength/maxLength</code> , both of which strict mode rejects.
<code>single_field_schema(field) · get_schema(name)</code>	For regenerating one field rather than the whole newsletter
NewsletterAppError + 30 subclasses	Every error carries a <code>user_message</code> (safe to display) separate from <code>message</code> (for logs). LLMRateLimitedError is distinct from LLMUnavailableError so "busy" and "broken" are handled differently.

6.2 config/

config/settings.py

Settings	Root object with 8 sections: app, logging, llm, email, scraper, database, brand, agent
<code>get_settings() -> Settings</code>	The accessor everything uses. Call it per operation, never cache the result — live settings edits mutate this object in place, and a cached copy would silently ignore them.
<code>build_settings(env_file=...) -> Settings</code>	Test seam. Passing None ignores the developer's .env , which is why the suite passes identically in CI.
<code>Settings.next_send_after(approved_at) -> datetime</code>	The scheduling calculation from §5.2
<code>Settings.describe_schedule() -> str</code>	Plain-English rendering for the dashboard
<code>env_var_name(section, field) -> str</code>	Maps a settings field back to its environment variable, for the Settings page

config/logging_config.py

<code>configure_logging(level)</code>	One-shot. Console + rotating JSONL file. Detects Windows colour support, because ConsoleRenderer(colors=True) raises SystemError on Windows without colorama.
<code>set_log_level(level)</code>	Moves only the console handler. The file sink stays at DEBUG deliberately — the forensic trail is what you want after something breaks, and it is no use if it was quiet at the time.
<code>redact_secrets(...) · mask_email(...)</code>	Processors that strip API keys and partially mask addresses before anything is written
<code>new_correlation_id() · bind_correlation_id(id)</code>	One id threads a whole generation or send through every log line

6.3 modules/ — the capability layer

Nine packages. Each hides exactly one external thing behind an interface, and each interface has a factory. **Every swap described in §9 happens inside this layer.**

modules/discovery/ — finding blog posts (M10)

DiscoverySource (base.py)	The interface: <code>fetch() -> list[DiscoveredPost]</code>
WordPressSource	Primary. Calls <code>/wp-json/wp/v2/posts</code> . Preferred because it returns a stable post id , which makes deduplication reliable even if a URL slug changes.
FeedSource	RSS fallback for non-WordPress blogs. Guards against a mis-served HTML page (_reject_doctype) and caps the download at MAX_FEED_BYTES .
<code>build_sources(settings) · discover_posts(settings)</code>	The factory and the one call the agent makes. Tries each source in order.

modules/scraper/ — getting article text

ArticleExtractor	Runs the three tiers in order and records which one won
-------------------------	---

<code>TrafilaturaExtractor</code>	Tier 1 — highest published F1. Succeeded on 100% of genuine articles in live measurement.
<code>NewspaperExtractor</code>	Tier 2 — Newspaper4k
<code>FallbackExtractor</code>	Tier 3 — BeautifulSoup heuristics. Last resort before the manual-paste message.
<code>ArticleFetcher</code> · <code>fetch_url(url)</code>	HTTP with a polite user agent, timeouts, retries and a robots.txt check

MEASURED, NOT ASSUMED

On eight real OEM articles (AWS, Cisco, Dell): 7/8 extracted, all at tier 1. The miss was an author-archive page, correctly refused with the manual-paste message. Fortinet's robots.txt disallow, a Microsoft 403 and an HPE timeout each produced their intended actionable message.

modules/cleaner/ – fitting the token budget

<code>TextCleaner</code>	Normalises whitespace and Unicode, removes boilerplate, dedupes repeated blocks
<code>estimate_tokens(text) -> int</code>	Pure-Python heuristic. This is what keeps torch (~2 GB) out of the dependency tree — D-14.
<code>truncate_to_budget(text, budget) -> TruncationResult</code>	Cuts at a paragraph boundary and reports how much was dropped

modules/ai/ – generation

<code>LLMProvider</code> (base.py)	The seam. <code>complete()</code> , <code>health_check()</code> , and the property <code>supports_guided_json</code> . This interface is why the Colab→Groq migration cost one class and a config default.
<code>GroqProvider</code> / <code>HostedProvider</code>	Both thin subclasses of the OpenAI-compatible base; they differ only in defaults
<code>OpenAICompatibleProvider</code>	The real work: strict schema requests, retry with backoff, Retry-After handling, error mapping
<code>MockProvider</code> / <code>FailingMockProvider</code>	Deterministic fixtures. Offline development and most tests. The failing variant provokes each error path.
<code>CircuitBreaker</code>	Opens after N consecutive failures so a dead endpoint fails fast rather than hanging every request; half-opens after a reset interval
<code>AIEngine</code>	The two-stage pipeline: <code>summarise_article()</code> per article, then <code>compose_newsletter()</code> once. Also <code>regenerate_field()</code> and <code>subject_variants()</code> .
<code>PromptRegistry</code> · <code>get_registry()</code>	Loads versioned YAML, renders Jinja2. <code>render()</code> takes positional-only parameters — a prompt variable named name collided with the method parameter, which is plausible in real use and baffling when it happens.
<code>create_provider(settings)</code> · <code>available_providers()</code>	The factory — the single line that decides which LLM you are using

modules/template/ – rendering the email

TemplateRenderer

Jinja2 → HTML → CSS inlining. Enforces compliance at render time: no unsubscribe link or postal address means no output.

`build_plain_text(draft) -> str`

Composed from the content fields directly, not converted from HTML — that is how `html2text` (GPL-3.0) was dropped (D-16), leaving every dependency permissively licensed

`strip_markdown(text) · apply_merge_tokens(...)`

Safety net for stray markdown; per-recipient personalisation tokens

`resolve_brand(settings) -> ResolvedBrand`

Name, colour, address, website

`resolve_logo_url(...).logo_file(...)`

Returns `cid:vays-logo` for a local file, or passes an `http(s)` URL through unchanged — so moving to a hosted logo needs no code change

modules/email/ – delivery

`EmailProvider` (base.py)

The seam: `send()`, `send_test()`, `health_check()`

`BrevoEmailProvider`

API provider. 300/day permanent free tier.

`SMTPEmailProvider`

Gmail and any ordinary SMTP host. Currently in use.

`ConsoleEmailProvider`

Writes `.eml` files to `data/outbox/` and sends nothing. **The safe default.**

`build_mime_body(...)`

Builds `multipart/related` with the logo attached *under* the HTML alternative. Attaching at the top level instead turns it into an ordinary attachment and the `cid:` resolves to nothing. Shared by console and SMTP so the preview and the real send cannot drift.

`unsubscribe_headers(...)`

List-Unsubscribe — deliverability, not decoration

`BatchSender`

Batches, paces, retries per recipient

`create_email_provider(settings)`

The factory — **the single line that decides how mail leaves**

modules/repository/ – persistence

Base + 12 ORM classes

`orm_models.py` — the whole schema in one readable file

`unit_of_work()`

Context manager: commit on success, rollback on exception. **Every write goes through this.**

`init_database() · get_engine()`

Sets WAL mode and `foreign_keys=ON` — both verified live, both off by default in SQLite

`CampaignRepository`

Includes `begin_send()`, the conditional UPDATE that is guard 2

<code>DiscoveredPostRepository · dedupe_key(post)</code>	Prefers <code>external_id</code> over URL. A unique constraint is the duplicate guarantee — application logic is the optimisation, not the guarantee.
<code>SubscriberRepository</code>	The master mailing list. Import appends; removal deactivates rather than deletes, so history stays intact.
<code>ArticleRepository · url_fingerprint(url)</code>	Article dedupe by normalised URL
<code>RecipientRepository / SuppressionRepository</code>	Per-campaign recipients; the permanent unsubscribe list
<code>SendRepository, LogRepository, SettingsRepository, UserRepository</code>	Delivery records; app logs with pruning; the settings override table; accounts
<code>DatabaseLogHandler · attach_db_log_handler()</code>	Feeds the Logs page. A write failure disables the handler rather than raising on every subsequent log line — an unreachable database would otherwise turn one problem into a storm. Lives here, not in <code>config/</code> , because <code>config</code> cannot import the repository without a cycle.

modules/tunnel.py – public URLs

<code>detect_ngrok_url() -> str None</code>	Queries the local ngrok agent API at <code>127.0.0.1:4040</code>
<code>resolve_base_url(settings) -> str</code>	The decision: an explicit <code>AGENT_APP_BASE_URL</code> wins; <code>auto</code> tries ngrok; otherwise localhost. Setting a real domain here is the entire production migration for approval links.
<code>local_url(settings) -> str</code>	<code>http://localhost:{APP_PORT}</code>

6.4 services/ — orchestration

One service per workflow. No Streamlit imports anywhere in this directory — enforced by an import-linter contract, and the reason `agent_worker.py` can drive the same code with no UI present.

services/agent_service.py – the automation orchestrator

<code>AgentService.run() -> int</code>	One discovery cycle: the in-flight check, then discover → record → extract → clean → generate → request approval. Returns how many campaigns it created.
<code>AgentRunReport</code>	Structured outcome — discovered, skipped, generated, failed, <code>holding</code> — surfaced on the dashboard
<code>_in_flight() -> int</code>	Counts campaigns not yet sent. The value compared against <code>max_in_flight</code> ; see §5.3.

services/approval_service.py

<code>ApprovalService.issue(campaign_id) -> ApprovalLink</code>	Mints a token, stores its hash, returns the raw link
--	--

<code>.check(token) -> TokenCheck</code>	Validates without consuming — used to render the review page
<code>.approve(token) · .reject(token)</code>	Consume the token and move the campaign. Sets approved_at in UTC .
ApprovalNotifier	Renders <code>templates/internal/approval_request.html</code> and sends it via the ordinary email provider
<code>review_url(token, settings) -> str</code>	Built on <code>resolve_base_url()</code> , so it follows ngrok or a real domain automatically

UTC INVARIANT

`approved_at` is stored and compared in UTC. SQLite returns naive datetimes, so `_as_utc()` normalises on read. Without it every expiry comparison raised `TypeError: can't compare offset-naive and offset-aware datetimes` — which would have broken every approval click. Preserve this if you touch the file.

services/dispatch_service.py – the send gate

`DispatchService.run() -> DispatchReport` Finds due campaigns and sends them

`.due() -> list[DueCampaign]` The gate, in one condition: `status == APPROVED and now >= next_send_after(approved_at)`

Other services

SERVICE	RESPONSIBILITY
GenerationService	The orchestrator every generation flows through — manual and automated alike. 100% test coverage , via a stub engine so each failure mode can be provoked directly.
DeliveryService	The four guards, then BatchSender . Largest service (470 lines) because the failure handling is the point.
IngestionService	URLs → extracted, cleaned, persisted articles. Per-URL results so one bad link does not fail the batch.
CampaignService	Draft lifecycle, state transitions, history
SubscriberService · <code>parse_pasted_addresses()</code>	The master list: CSV import with an ImportReport , paste-a-list, deactivation
RecipientSource · <code>recipients_dir()</code> · <code>latest_csv()</code>	Resolves who a campaign goes to — the subscriber table, or a CSV drop directory
SettingsService	The 43-entry editable registry, validation, persistence, revert. See §8.
AuthService	Login, lockout counting, user CRUD, <code>set_password()</code> . Self-deactivation is refused, not warned about — the only admin locking themselves out has no way back in.

SERVICE	RESPONSIBILITY
<code>HealthService</code>	Real probes against the LLM and email providers. One cached instance shared by the sidebar and the dashboard panel, so they cannot disagree.
<code>agent_status.record()</code> / <code>current()</code>	Worker heartbeat — the dashboard shows "worker last seen", which is how you tell the worker is not running

6.5 `ui/` — Streamlit

PAGE	DOES
Dashboard <code>ui/pages/dashboard.py</code>	Health, agent status and next send time, recent campaigns with a working Open button (sets the draft id, then <code>st.switch_page("preview")</code>)
Generate Newsletter	Paste URLs → ingest → set tone/length/audience → generate
Campaign Preview	Edit every field, regenerate one field, subject variants, live HTML preview, send
Approvals	The page an approval link opens; also lists campaigns awaiting approval for in-app action
Recipients	The master list — CSV upload, paste, remove, counts
Campaign History	Past campaigns, per-recipient delivery outcomes, retry-failed
Settings	The 43 editable settings, Test Connection buttons, revert. Admin-only.
Logs	Searchable <code>app_logs</code> with level and correlation-id filters

SUPPORT MODULE	PUBLIC SURFACE
<code>ui/state.py</code>	<code>current_user()</code> , <code>set_current_user()</code> , <code>is_authenticated()</code> , <code>logout()</code> , <code>health_service()</code> — typed access to <code>st.session_state</code> with the keys in one place
<code>ui/session.py</code>	<code>remember()</code> , <code>forget()</code> , <code>stored_token()</code> , <code>restore()</code> — cookie-backed persistence so a refresh does not log you out. See §12.3.
<code>ui/styles.py</code>	<code>inject()</code> , <code>status_chip()</code> , <code>health_dot()</code> , <code>dashboard_logo()</code> — the LUNA palette lives here as constants
<code>ui/components.py</code>	10 shared widgets: metric cards, error panels, empty states, character counters, the send confirmation, provenance cards

7 Data model

Twelve tables, five Alembic migrations. SQLite in WAL mode with `foreign_keys=ON` — both set explicitly at connect time, because SQLite defaults to neither.

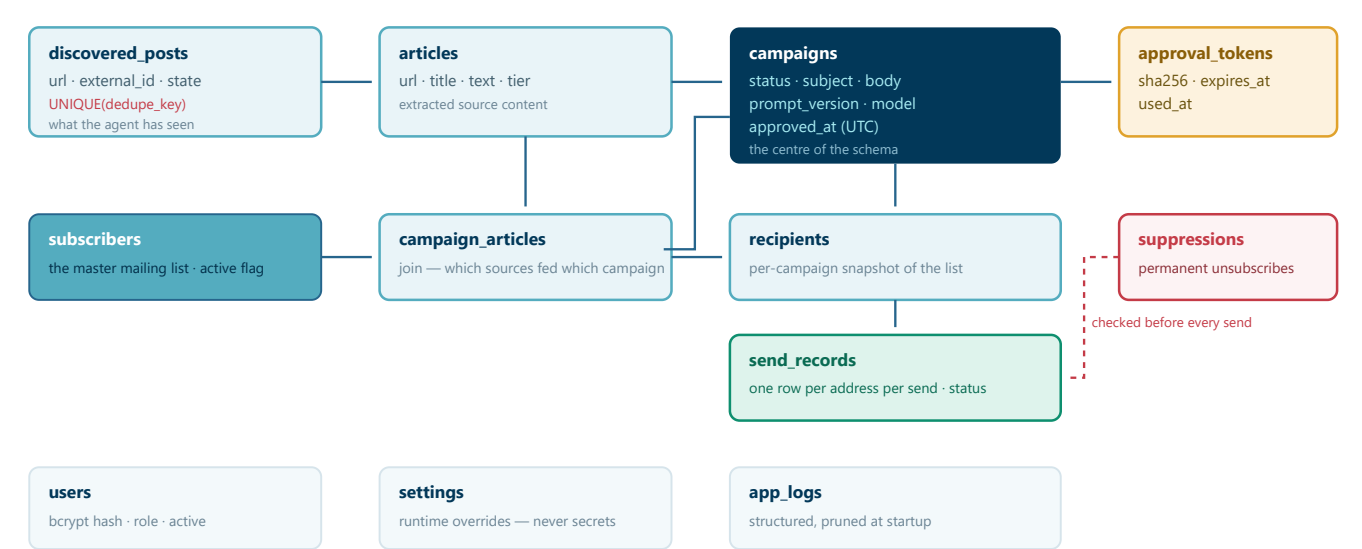


Figure 10 — Schema. `campaigns` is the centre. Note that `recipients` is a per-campaign *snapshot*, separate from `subscribers` (the living master list) — so history shows who a campaign actually went to, even after the list changes.

TABLE	PURPOSE AND THE ONE THING WORTH KNOWING
<code>campaigns</code>	Records <code>prompt_version</code> and <code>model</code> on every row, so any output is reproducible — D-6
<code>articles</code>	Deduped by normalised-URL fingerprint; stores which extractor tier succeeded
<code>campaign_articles</code>	Many-to-many, which is what makes the provenance panel possible
<code>discovered_posts</code>	The agent's memory. Clearing this table makes the agent re-process old posts — see §10.5.
<code>approval_tokens</code>	Hash only. Never the raw token.
<code>subscribers</code>	Removal sets <code>active = false</code> ; the row survives so past sends still resolve
<code>recipients</code>	Per-campaign snapshot
<code>send_records</code>	One row per address per send. This is what makes "retry failed only" possible without re-mailing successes.
<code>suppressions</code>	Permanent. Checked in bulk before every send, so an unsubscribe survives a re-uploaded CSV.
<code>settings</code>	Runtime overrides on top of <code>.env</code> . A <code>SecretStr</code> field cannot reach it — §8.2.
<code>users</code>	bcrypt hashes. No default account and no default password exist anywhere in the codebase.
<code>app_logs</code>	Feeds the Logs page. Pruned at startup so it cannot grow without bound.

MIGRATIONS

```
alembic upgrade head          # apply everything (this is what run.bat checks)
alembic revision --autogenerate -m "add x"  # then READ the generated file before applying
alembic downgrade -1          # step back one
alembic current                # what the database is on now
```

SQLITE AND ALTER TABLE

SQLite cannot drop or alter a column in place; Alembic emulates it by rebuilding the table. Autogenerate handles the common cases, but always read the generated migration before running it against a database with real campaign history — and take a copy of `data/newsletter.db` first. It is one file; copying it is the entire backup procedure.

8 Configuration

8.1 Two layers (D-24)

Configuration has a bottom layer and a top layer, and both matter for different reasons.

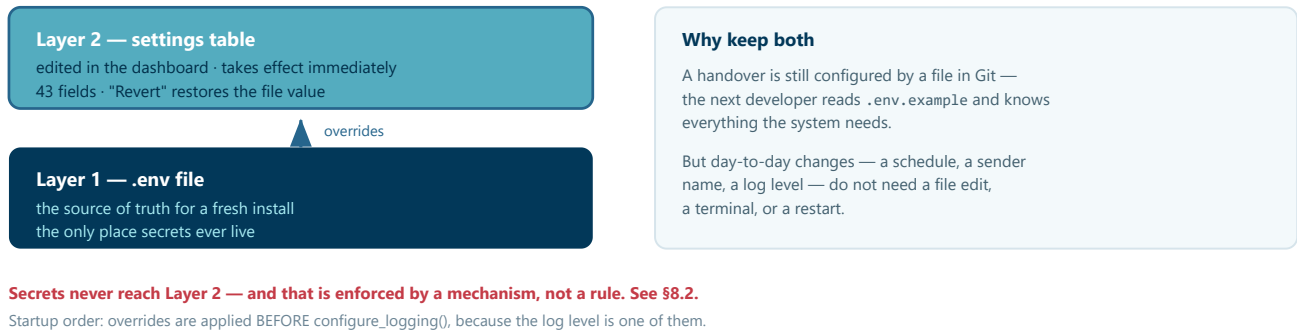


Figure 11 — The two configuration layers. A saved value that no longer validates is logged and skipped, never fatal. Refusing to boot over a value that can only be fixed in the UI you just prevented from loading is a trap with no exit.

8.2 The secret rule, as a mechanism

`SettingsService._validate_registry()` runs **at import time** and raises if any registered field is a `SecretStr`. Adding `llm.api_key` to the editable registry therefore fails the build, not code review. That converts D-19 ("secrets stay in `.env`") from a convention a future developer could break into an invariant they cannot.

Three further properties make live editing safe rather than merely convenient:

- **`validate_assignment=True`** on every settings section, so an edit runs the same validators as startup. A rejected value leaves the old one in place — the app is never left holding a config it could not have booted with. Verified: `temperature=99` is refused and 0.7 survives.
- **The stored value is the normalised one**, read back off the model after assignment. `https://host/v1/` persists as `https://host`. Storing raw input would leave the database and the running process disagreeing about the endpoint — on this page of all pages.
- **Live mutation, not a rebuild.** Every holder of `get_settings()` sees the change because the object is mutated in place. This only works because factories call `get_settings()` per operation instead of caching a provider — an invariant of *other* modules, so a test class asserts it. If someone later caches a provider, those tests fail rather than the feature silently doing nothing.

8.3 Environment variables

VARIABLE	DEFAULT	NOTES
Application		
<code>APP_ENV</code>	<code>dev</code>	<code>dev</code> · <code>prod</code>
<code>APP_SECRET_KEY</code>	<i>required</i>	secret Signs session tokens. Generate with <code>secrets.token_urlsafe(48)</code> . Changing it logs everyone out.
<code>APP_PORT</code>	<code>8501</code>	Used to build localhost links

VARIABLE	DEFAULT	NOTES
LOG_LEVEL	INFO	Console only; the file sink stays at DEBUG
DATABASE_URL	sqlite:///data/newsletter.db	Any SQLAlchemy URL — see §9.5
LLM		
LLM_PROVIDER	groq	groq · hosted · mock . colab now fails at startup by design.
GROQ_API_KEY	<i>required</i>	secret Starts gsk_ . CI greps full git history for this prefix.
LLM_MODEL	openai/gpt-oss-120b	Apache 2.0 open-weight despite the name — see §9.1
LLM_BASE_URL	https://api.groq.com	Normalised on save; trailing <code>/v1/</code> is stripped
LLM_MAX_TOKENS	2048	Must be ≥ 2048 for structured output. See the warning below.
LLM_TEMPERATURE	0.85	Raised from 0.7 for a less machine-like voice; the banned-construction list keeps the variance in check
LLM_TIMEOUT_S · LLM_MAX_RETRIES	60 · 3	
LLM_CIRCUIT_FAILURE_THRESHOLD · LLM_CIRCUIT_RESET_S	5 · 60	Circuit breaker
Email		
EMAIL_PROVIDER	console	console · smtp · brevo . console sends nothing — the safe default.
EMAIL_SENDER_NAME · EMAIL_SENDER_ADDRESS · EMAIL_REPLY_TO	—	The From/Reply-To identity
EMAIL_BATCH_SIZE · EMAIL_BATCH_DELAY_S · EMAIL_MAX_RETRIES	50 · 1 · 3	Pacing, to stay inside provider limits
BREVO_API_KEY	—	secret Starts xkeysib-
SMTP_HOST · SMTP_PORT · SMTP_USERNAME · SMTP_USE_TLS	— · 587 · — · true	Gmail: smtp.gmail.com , 587, TLS on
SMTP_PASSWORD	—	secret For Gmail this is a 16-character App Password , not the account password
Agent (M10)		
AGENT_ENABLED	false	Master switch. The worker exits immediately when false.
AGENT_BLOG_URL	https://vaysinfotech.com	Discovery target
AGENT_DISCOVERY_INTERVAL_HOURS	6	1–168. Does not control send frequency.

VARIABLE	DEFAULT	NOTES
AGENT_MAX_IN_FLIGHT	1	This is what enforces one newsletter per month. \$5.3.
AGENT_MAX_POSTS_PER_RUN	1	Per-run cap; not sufficient alone
AGENT_SEND_SCHEDULE	monthly	monthly · daily
AGENT_SEND_WEEKDAY · AGENT_SEND_WEEK_OF_MONTH · AGENT_SEND_TIME	wednesday · 3 · 11:00	Anchored HH:MM regex — 9:00 is rejected, 09:00 is correct
AGENT_TIMEZONE	Asia/Kolkata	IANA name
AGENT_APPROVAL_EMAIL	—	Where the approval request goes
AGENT_APPROVAL_TOKEN_TTL_HOURS	72	1–720
AGENT_APP_BASE_URL	auto	auto → try ngrok, else localhost. Set a real domain here in production.
AGENT_RECIPIENTS_DIR	data/recipients	CSV drop directory
NGROK_DOMAIN	—	A reserved ngrok domain, if you have claimed one
Brand & scraping		
BRAND_NAME · BRAND_PRIMARY_COLOR · BRAND_WEBSITE	Vays Infotech · #54ACBF · —	Rendered into every template
BRAND_LOGO_PATH	assets/logo.png	Local file → CID-embedded. An http(s) URL is passed through unchanged.
BRAND_ADDRESS	<i>required to send</i>	Postal address — guard 4 refuses to render without it
UNSUBSCRIBE_BASE_URL	—	Also required by guard 4
SCRAPER_MAX_INPUT_TOKENS	3000	Cut from 6000 because of the Groq free-tier TPM ceiling — risk C-8
SCRAPER_TIMEOUT_S · SCRAPER_MAX_RETRIES · SCRAPER_MAX_CONCURRENT	20 · 2 · 3	
SCRAPER_RESPECT_ROBOTS	true	Leave it true
SCRAPER_MIN_WORD_COUNT	150	Below this, extraction is treated as failed
SCRAPER_USER_AGENT	identifying UA	Polite and identifiable, not a browser impersonation

LLM_MAX_TOKENS MUST BE ≥ 2048

`gpt-oss` emits internal reasoning tokens that count against the budget but never appear in the response. A budget sized to the visible output is cut off mid-JSON, and

GROQ REPORTS THAT AS A 400 JSON_VALIDATE_FAILED, NOT FINISH_REASON: LENGTH

— so the token-budget error path never fires and the failure looks like a schema problem.

`scripts/validate_prompts.py` enforces the floor. This costs real debugging time; do not lower it.

9 Swapping things out

Everything below was designed to be replaceable, and most of these have already been done once for real. Each entry gives the reason you would do it, the exact steps, and what to run afterwards to know it worked.

9.1 The LLM model

1 config line

Why: cost, quality, a faster model, or replacing `gpt-oss-120b` if Groq retires it.

```
LLM_MODEL=openai/gpt-oss-20b      # smaller, faster, cheaper on TPM
# or a Qwen / Llama model Groq serves
```

The one trade-off. Only the `gpt-oss` family supports `strict: true` schema enforcement, which is what makes D-3 a guarantee rather than a hope. Switching to a Qwen or Llama model still works — `supports_guided_json` becomes `False` and the provider falls back to a repair-retry loop — but JSON validity becomes probabilistic instead of guaranteed. Weigh that before switching.

Verify: Settings → Test Connection (expect healthy plus a latency figure), then generate one real newsletter and confirm it validates on the first attempt.

9.2 The LLM host

1 class + 1 default

Why: moving off Groq entirely — to Together, Fireworks, an internal vLLM server, or anything OpenAI-compatible.

1. If it is OpenAI-compatible, set `LLM_PROVIDER=hosted` and point `LLM_BASE_URL` at it. **No code at all.**
2. If it is not, add a class in `modules/ai/` implementing `LLMProvider` (three members: `complete()`, `health_check()`, `supports_guided_json`), then register it in `modules/ai/factory.py::create_provider()`.
3. Add the literal to `LLMSettings.provider` in `config/settings.py`.

This has been done for real. The Colab→Groq migration (D-21) cost exactly one class and one config default — no service, page, or test outside `modules/ai/` changed. That is the strongest evidence the seam is real.

Verify: `pytest tests/unit/test_ai_providers.py`, then Test Connection, then one live generation.

9.3 The email provider

1 config line

Why: Gmail's ~500/day limit, moving to a real ESP, or going quiet for testing.

<code>console</code>	Writes <code>.eml</code> to <code>data/outbox/</code> . Sends nothing. Use for any test.
<code>smtp</code>	Gmail or any SMTP host. Gmail needs an App Password , not the account password.
<code>brevo</code>	API. 300/day permanent free tier; combines transactional and marketing.

For a provider that is none of these: implement `EmailProvider` in `modules/email/` and register it in `create_email_provider()`. Reuse `build_mime_body()` so the CID logo keeps working.

ASSERT THE PROVIDER IN ANY SCRIPT THAT CAN SEND

A script of mine assumed `EMAIL_PROVIDER=console` without checking. `.env` had been switched to `smtp`, so three "preview" calls went to Gmail for real. The address was RFC-2606 undeliverable so nobody received anything, but it consumed quota and generated bounces. Every send-capable script now asserts the provider first. Keep that habit.

9.4 Email templates

new HTML file

Why: a rebrand, a new layout, or a seasonal variant.

1. Copy `templates/email/classic.html` to a new name in the **same directory**.
2. Keep the Jinja2 variables and the compliance block — the unsubscribe link and postal address are what guard 4 checks for. Removing them means the template can never send.
3. Use table-based layout with inline-safe CSS. Node is not installed, so MJML was dropped (D-23); hand-authored table HTML is the source of truth.
4. It appears in the Preview page picker automatically — the picker lists the directory.

DO NOT PUT NON-NEWSLETTER TEMPLATES IN `TEMPLATES/EMAIL/`

`approval_request.html` was placed there and, sorting before `classic`, silently became the *default* newsletter template and broke the Preview page. Internal templates live in `templates/internal/` for exactly this reason.

9.5 The database

1 URL + driver

Why: more than one concurrent writer, a hosted deployment, or backup requirements SQLite cannot meet.

```
pip install psycopg[binary]
DATABASE_URL=postgresql+psycopg://user:pass@host:5432/vays_newsletter
alembic upgrade head           # builds the schema fresh on Postgres
```

Everything is SQLAlchemy 2.x with no raw SQL and no SQLite-only types, so the ORM layer carries over unchanged. Two things to check:

- The WAL and `foreign_keys` pragmas in `database.py` are SQLite-specific. They are applied on connect and are harmless elsewhere, but read that function before you move.
- Migrating *existing data* is a separate exercise from creating the schema. For a fresh Postgres, `alembic upgrade head` is genuinely all of it.

Verify: the full integration suite runs against the new URL — `pytest tests/integration`.

9.6 The blog source

1 class

Why: Vays moves off WordPress, or you want to monitor OEM partner blogs instead of (or as well as) the Vays blog.

1. Point `AGENT_BLOG_URL` elsewhere. WordPress REST is tried first, RSS second — most blogs need nothing more.
2. For a site with neither: implement `DiscoverySource` in `modules/discovery/` — one method, `fetch()` — `> list[DiscoveredPost]` — and add it to `build_sources()`.

Always populate `external_id` if the source has a stable identifier. `dedupe_key()` prefers it over the URL, so a changed slug does not resurface a post that has already been sent.

Verify: `python agent_worker.py --once` with `EMAIL_PROVIDER=console`, then check the `discovered_posts` table.

9.7 Public hosting for approval links

1 env var

Why: today the approval link points at `localhost` or a rotating ngrok URL, so it only works from the machine running the dashboard. This is the change that makes it work from a manager's phone.

```
# development – no configuration, tries ngrok then falls back
AGENT_APP_BASE_URL=auto

# production – one line, and every generated link follows
AGENT_APP_BASE_URL=https://newsletter.vaysinfotech.com
```

Everything that builds a link routes through `resolve_base_url()`, so this single variable is the whole migration. Nothing else needs to change.

CURRENT STATE

ngrok is installed but has

NO AUTHTOKEN CONFIGURED

and no reserved domain, so its URL rotates on every restart. Claiming a free static domain is worth ten minutes if approval-from-phone matters before a real domain exists.

9.8 Authentication

additive

Why: company SSO, or moving off self-managed accounts.

The documented upgrade path is to replace `AuthService.authenticate()` with an OIDC/SAML exchange while keeping the rest of the interface. `UserRole` and the `users` table stay as the authorisation layer; only the identity check changes.

D-15 chose ~160 auditable lines in `core/auth.py` over an unvetted third-party package on the security boundary, precisely so this file could be read in full at handover and replaced deliberately.

9.9 Prompts

[edit YAML](#)

Why: tone, length, structure, or fixing something the model keeps getting wrong.

Prompts are versioned, never edited in place (D-6). Copy `prompts/newsletter_compose/v1.1.0.yaml` to `v1.2.0.yaml` and change that. Every campaign records the prompt version that produced it, so old campaigns still resolve their original prompt and remain reproducible.

Shared fragments live in `prompts/_shared/`: `human_voice.md`, `length_rules.md`, `system_persona.md`, `untrusted_input_rules.md`. Editing one changes every prompt that includes it — which is usually what you want for a voice change.

TWO LESSONS THAT COST REAL DEBUGGING

- **State limits as hard limits.** "approximately 60 characters" overshot every time. `MAXIMUM 60 characters` (this is a hard limit, not a target) landed in range on the first try.
- **A prompt instruction is not a guarantee.** v1.0.0 said "give each story a short bold heading"; the field is plain text, so the model emitted `**Heading**` and the asterisks reached the customer. Fixed at both ends — v1.1.0 bans markdown explicitly, and `_split_paragraphs` converts a whitelist of markdown to real HTML as a safety net. **Order is the security property:** each paragraph is escaped first, then our own tags are inserted into the escaped string, so the safety net cannot become an injection hole.

Verify: `python scripts/validate_prompts.py` (checks schema and the `max_tokens` floor), then generate against a real article and read the output.

9.10 Swap summary

SWAP	EFFORT	TOUCH POINTS	PROVEN?
LLM model	trivial	LLM_MODEL	yes — 120b and 20b both run
LLM host	1 class	<code>modules/ai/factory.py</code>	yes — Colab → Groq
Email provider	trivial	EMAIL_PROVIDER	yes — console, SMTP live
Template	1 file	<code>templates/email/</code>	yes — 3 ship
Database	1 URL + driver	DATABASE_URL	designed for; not exercised
Blog source	1 class	<code>modules/discovery/</code>	yes — WordPress + RSS
Public URL	1 var	AGENT_APP_BASE_URL	yes — localhost and ngrok
Auth	1 method	<code>AuthService.authenticate</code>	designed for; not exercised
Prompts	new YAML	<code>prompts/</code>	yes — v1.0.0 → v1.1.0, A/B measured
Add a FastAPI shell	additive	new package calling <code>services/</code>	the zero-Streamlit-import rule is enforced today

10 Operations runbook

10.1 Starting and stopping

COMMAND	STARTS
<code>run.bat</code>	The dashboard on <code>http://localhost:8501</code> . Preflight-checks venv, <code>.env</code> , migrations and accounts.
<code>run_agent.bat</code>	The agent worker. Without this, nothing is automatic.
<code>run_tunnel.bat</code>	ngrok, so approval links work off-machine.
<code>python agent_worker.py --once</code>	One discovery+dispatch cycle, then exit. The safe way to test.
<code>install_agent_task.bat</code>	Registers the worker as a Windows scheduled task so it survives a reboot.

Stop either process with `Ctrl+C` in its window. Closing the dashboard does not stop the worker, and stopping the worker does not affect anyone using the dashboard — they are independent (§3.3).

10.2 The single most common problem

"THE AGENT ISN'T DOING ANYTHING"

In every instance so far, the worker process was simply not running. `run.bat` starts the

DASHBOARD ONLY

. Check the Dashboard's agent panel: if

WORKER LAST SEEN

reads *never* or a stale timestamp, start `run_agent.bat`. Then confirm `AGENT_ENABLED=true` — the worker exits immediately when it is false.

10.3 Diagnosis table

SYMPTOM	MOST LIKELY CAUSE	FIX
No campaigns appear automatically	Worker not running, or <code>AGENT_ENABLED=false</code>	<code>run_agent.bat</code> ; check the dashboard heartbeat
Campaign sits in <code>AWAITING_APPROVAL</code>	Working as designed — nobody has approved	Approve via the email link or the Approvals page
Approved but not sending	Not yet the 3rd Wednesday	Read "next send" on the dashboard. This is normal (§5.2).
Approval link 404s / does not open	ngrok restarted and rotated its URL, or the link points at localhost from another device	Set <code>AGENT_APP_BASE_URL</code> (§9.7)
Approval link says invalid	Token used, expired, or for a different campaign — all four checks fail identically	Approve from the Approvals page instead
Sidebar says AI service offline	Groq unreachable, bad key, or the circuit breaker is open	Settings → Test Connection for the real error

SYMPTOM	MOST LIKELY CAUSE	FIX
Generation fails with <code>json_validate_failed</code>	<code>LLM_MAX_TOKENS</code> below 2048	Raise it — \$8.3 warning
Generation slow or 429	Groq free-tier TPM ceiling (risk C-8)	<code>Retry-After</code> is honoured automatically; reduce <code>SCRAPER_MAX_INPUT_TOKENS</code> or use <code>gpt-oss-20b</code>
Nothing arrives in an inbox	<code>EMAIL_PROVIDER=console</code>	Check <code>data/outbox/</code> — the emails are there as <code>.eml</code>
Send refuses outright	Guard 4 — no unsubscribe URL or postal address	Set <code>BRAND_ADDRESS</code> and <code>UNSUBSCRIBE_BASE_URL</code>
Logo missing in the email	<code>BRAND_LOGO_PATH</code> points at a missing file	Degrades to a text fallback by design; fix the path. Never blocks a send.
Refresh logs you out	Cookie blocked, or <code>APP_SECRET_KEY</code> changed	Changing the key invalidates every session — expected
Multiple newsletters at once	<code>AGENT_MAX_IN_FLIGHT</code> above 1	Set it to 1 — \$5.3

10.4 Where to look

Logs page	Searchable, filterable by level and correlation id. Start here.
<code>logs/app.jsonl</code>	Rotating JSON lines, always at DEBUG regardless of the console level
Dashboard agent panel	Worker heartbeat, last run report, next send time
Campaign History	Per-recipient delivery status and failure reasons
<code>data/outbox/</code>	<code>.eml</code> files when the provider is <code>console</code>

A correlation id threads one generation or send through every log line, so filtering on it gives the complete story of a single operation.

10.5 Routine tasks

BACK UP

```
copy data\newsletter.db backups\newsletter-2026-08-18.db
```

SQLite is one file. Copy it. In WAL mode, prefer copying while nothing is writing, or use `sqlite3 data/newsletter.db ".backup backups/x.db"` for a consistent snapshot.

MAKE THE AGENT FORGET DISCOVERED POSTS

Clearing `discovered_posts` makes the agent treat every existing blog post as new. That is occasionally what you want (a fresh start) and usually not (it will regenerate old content). Take a backup first, and remember `max_in_flight` is what stops a re-discovered backlog becoming a flood of email.

CHANGE WHO RECEIVES NEWSLETTERS

Recipients page: upload a CSV (appends — it does not replace), paste addresses, or remove individuals. Removal deactivates rather than deletes, so past campaign history still resolves. Suppressed addresses stay suppressed even if re-uploaded.

CHANGE WHO APPROVES

`AGENT_APPROVAL_EMAIL` , or Settings → Agent. Takes effect on the next approval request; already-issued tokens are unaffected.

11 Testing & quality gates

Six gates. All six must pass before anything is committed, and CI runs them on Windows and Ubuntu across Python 3.11 and 3.12.

GATE	STATUS	PROTECTS
<code>python scripts/check_no_local_inference.py</code>	pass	The zero-local-inference guarantee. Runs first in CI — if someone adds torch, nothing else matters.
<code>ruff format --check .</code>	pass	Formatting
<code>ruff check .</code>	pass	Lint
<code>mypy core config modules services</code>	74 files clean	Types
<code>lint-imports</code>	4 kept, 0 broken	The layer rule (§3.1)
<code>pytest</code>	1,168 passed · 93%	Behaviour

```
# everything, in the order CI runs it
python scripts/check_no_local_inference.py
ruff format --check . && ruff check .
mypy core config modules services
lint-imports
pytest

# faster loops
pytest tests/unit -q           # pure logic, no I/O
pytest tests/integration -q    # real SQLite, stubbed network
pytest tests/e2e -q            # Streamlit AppTest – renders every page
pytest --cov --cov-report=html # htmlcov/index.html
```

11.1 Test principles that were learned, not assumed

- **A coverage percentage hides a zero.** `generation_service.py` — the orchestrator every generation flows through — had **0% coverage** while the suite reported 90% overall. It is now 100%, tested with a stub engine rather than `MockProvider` so each failure mode can be provoked directly. This was the **fourth** instance of the same dead-code/untested-core pattern (`app_logs` , `settings` , `set_password` , then this). **Grep for repositories with no production caller before assuming a module works.**
- **Synthetic fixtures do not exercise token budgets.** `article_summary` at 1024 tokens passed on a synthetic article and failed on a real one.
- **Tests must not touch the network or real data.** Two tests were caught hitting the live blog and the real Groq API, and one read the developer's actual recipients directory. Fixtures now stub discovery and point at temp directories.
- **A test must discriminate the bug.** My first health-check test passed against both the broken and fixed code because the mock provider is healthy either way. Rewritten around "configured but unhealthy", which is the

case that actually differed.

- **SQLite returns plain `str` for enum columns.** `StrEnum` compares equal but not identical, so assert with `==`, never `is`.
- **`AppTest.from_function` re-executes the function's source**, so a closure over an outer variable is silently lost — pass values via `args=`. The first `test_pages.py` asserted on pages it never rendered and would have passed against a UI that crashed on every click.

11.2 CI details worth keeping

- `pip-audit` is `continue-on-error` and therefore **advisory** — a green tick does not mean clean. Read its output.
- A dedicated job greps **full git history** for `gsk_` and `xkeysib-`, because a secret deleted in a later commit is still leaked. Verified: history is clean and `.env` was never tracked.
- The suite is also run under CI conditions (no `.env`, environment variables only) to prove the workflow configuration is right rather than plausible.
- Every internal documentation link is checked by a script.

12 Security

12.1 Secrets

CONTROL	WHERE
<code>.env</code> is git-ignored; <code>.env.example</code> is committed	Verified — <code>.env</code> was never tracked
Secrets cannot enter the editable settings registry	Import-time <code>SecretStr</code> check — §8.2
Secrets are redacted from every log line	<code>redact_secrets</code> processor; addresses partially masked
Git history scanned for key prefixes	Dedicated CI job over full history
Passwords are bcrypt with per-password salts	<code>core/auth.py</code>
No default account or password exists	First login is created by <code>python -m scripts.create_user</code> , which enforces strength. There is no fallback credential to forget to change.

12.2 Input handling

- **SSRF guard on every URL.** `resolve_public_addresses()` refuses loopback, link-local and RFC-1918 ranges — so a pasted `http://169.254.169.254/` cannot make the server fetch cloud metadata.
- **Scraped text is untrusted input.** `prompts/_shared/untrusted_input_rules.md` instructs the model to treat article content as data, not instructions.
- **Escape first, then insert.** The markdown safety net escapes each paragraph before inserting our own tags into the escaped string, so it cannot become an HTML-injection hole.
- **Approval tokens** — random, hashed at rest, single-use, expiring, campaign-scoped (§4.3).

12.3 Sessions

A browser refresh starts a new Streamlit session, so `session_state` is empty even though nobody signed out. `ui/session.py` restores the session from a cookie carrying the same HMAC-signed token the login issues. Three checks run before it is honoured:

1. **The signature verifies** against `APP_SECRET_KEY`. A forged or tampered cookie is indistinguishable from a missing one.
2. **The token has not expired** — 12 hours, carried inside the payload.
3. **The account still exists and is active.** This is the check the token cannot make for itself: without it, deactivating someone would leave them signed in for up to twelve more hours.

The role that is honoured comes from the database, not from the token, so a stale cookie cannot preserve a privilege that has since been revoked. A cookie that fails any check is cleared, so the browser stops presenting something that will never work again. Eleven tests cover these paths.

A DELIBERATE DISTINCTION FROM D-15

`extra-streamlit-components` (Apache-2.0) is used purely as cookie *storage* — it writes and reads an opaque string. Every security property stays in `core/auth.py`. That is why this is compatible with D-15, which refused `streamlit-authenticator` because that package would have owned authentication itself.

A COOKIE JAR OWNS NOTHING.

KNOWN LIMITATION

The cookie is written from the browser side and therefore cannot be `HttpOnly`. That is inherent to setting cookies from Streamlit, not a shortcut. It is acceptable on a LAN-internal tool;

IF THIS APP IS EVER EXPOSED PUBLICLY, REVISIT IT

— the mitigation is a server-side session store, or terminating auth at a reverse proxy.

12.4 Roles

Two roles. **Editor** can generate, edit and send. **Admin** adds settings editing and user management. Password change re-authenticates first, so it runs the lockout counter and cannot be used to brute-force from inside a session. Self-deactivation is refused rather than warned about — the only admin locking themselves out has no way back in.

13 Decision register

The full register is `docs/09_FINAL_DECISIONS.md` with ADRs for the reversals. Summarised here because the reversals are the ones a future developer would otherwise re-litigate.

13.1 Decisions that held

ID	DECISION	WHY IT STILL STANDS
D-1	No separate FastAPI backend in v1	One deployable is simpler to hand over. The service layer has zero Streamlit imports, so adding an API later is additive, not a rewrite.
D-2	"Open source" constrains the model , not the host	This is what makes Groq legitimate: it serves only open-source models.
D-3	Guided JSON decoding, not prompt-and-pray	strict: true constrains generation to the schema, so json.loads cannot fail.
D-4	Two-stage generation	Smaller prompts give better quality on a mid-size model and cheaper retries.
D-5	Human approval is mandatory	The core mitigation for the model inventing an OEM fact. Not removable by configuration.
D-6	Prompts are versioned YAML in Git	Every campaign records the version that produced it, so any output is reproducible.
D-12/13	No local inference, machine-enforced	A CI gate, not a code-review comment.
D-14	Token counting is a pure-Python heuristic	Keeps a 2 GB dependency out of the tree.
D-15	Own auth module	No unvetted package on the security boundary; auditable at handover.
D-16	html2text (GPL-3.0) dropped	Every remaining dependency is permissively licensed.
D-19	Secrets stay in .env	Enforced by an import-time mechanism, not a rule.

13.2 Decisions that were reversed — read these first

ID	REVERSAL	WHAT ACTUALLY HAPPENED
D-21	Colab dropped; Groq adopted	Colab was attempted twice on real hardware and never generated a token — a CUDA-13 vs CUDA-12 wheel mismatch, then an install/restart loop. Beyond those fixable faults the <i>shape</i> was wrong: 3-hour sessions, a rotating tunnel URL, no guaranteed GPU, and a standing ToS conflict. The switch cost one class and a config default , which is the clearest possible validation of the LLMProvider seam.
D-23	MJML dropped	Node is not installed, so .mjml sources could not be compiled. An uncompileable source of truth is worse than none. Hand-authored table HTML removes the last Node dependency at any stage.
D-24	Two-layer configuration	Runtime settings override .env , which stays the bottom layer so a handover is still configured by a file in Git.

ID	REVERSAL	WHAT ACTUALLY HAPPENED
CID logo	Filesystem path → <code>cid:</code>	The original code put a <i>filesystem</i> path into <code></code> , which no mail client can resolve. Gmail and Outlook both strip <code>data:</code> URIs, so CID embedding is the only approach that works without web hosting.

13.3 Live risks

ID	RISK	MITIGATION
C-8	Groq free-tier token limit (~8–12k TPM) — the binding constraint	Input budget cut 6000 → 3000 per article; Retry-After honoured; LLMRateLimitedError distinguishes "busy" from "broken". A paid tier is ~10× if it bites.
C-9	openai/gpt-oss-120b looks proprietary and is not	Apache 2.0 open-weight. Documented in 04_LLM_HOSTING.md §4 and .env.example , because this will be asked at review.
C-3	Model hallucinates or fabricates an OEM claim	Mandatory human review; source-URL provenance shown beside each block; grounding instructions; an extractive summary stage before the generative one.
C-4	Blog sites block scraping or are JS-rendered	Three-tier cascade, polite UA, robots.txt check, manual-paste fallback.
C-5	Email lands in spam	List-Unsubscribe , plain-text alternative, verified sender domain (SPF/DKIM/DMARC), suppression list.
C-6	Copyright — republishing OEM content	Original summaries, always link back, attribution block in the template.

14 Known gaps & next steps

Stated plainly, because a handover that overstates completeness is worse than one that admits a gap.

GAP	SEVERITY	DETAIL AND WHAT CLOSING IT TAKES
M5 — visual email-client verification	open	Templates render correctly and CSS inlines, but nobody has opened the result in real Outlook and Gmail . This needs a human with those clients — it cannot be automated here. Open the <code>.eml</code> files in <code>data/outbox/</code> and check the logo, the header band and mobile width. This is the last M5 item and the reason M5 is amber, not green.
ngrok has no authtoken	open	The URL rotates on every restart, so approval links go stale. Claim a free static domain, or set <code>AGENT_APP_BASE_URL</code> to a real domain (§9.7).
Recipient list is test addresses	before go-live	Currently two personal addresses. Replace with the real customer list on the Recipients page before enabling the agent.
Agent is disabled	intentional	<code>AGENT_ENABLED=false</code> by design until the list and approval routing are final.
Postgres path untested	low	No SQLite-only SQL is used, but the migration has not been exercised. Do it before assuming it is free.
Session cookie is not <code>HttpOnly</code>	if public	Inherent to setting cookies from Streamlit. Acceptable on a LAN; revisit before public exposure (§12.3).
Groq free-tier ceiling	watch	One article ≈ 3,450 tokens against ~8–12k per minute. Fine at one post per month; it would bite immediately at higher volume.
Handover walkthrough with Vays	scheduled	This document is the material for it.

14.1 If you are the next developer, do this first

1. Run the quick start in §2. If it does not work from this document alone, that is a documentation bug worth reporting.
2. Run all six gates (§11). They should be green before you change anything, so you know any later failure is yours.
3. Read `core/models.py` and `core/enums.py` — about 700 lines that describe every object in the system.
4. Run `python agent_worker.py --once` with `EMAIL_PROVIDER=console` and watch the logs. That is the whole automated pipeline in one command, with nothing leaving the machine.
5. Read §13.2 before reversing any decision. Each of those was reversed once already, for reasons that are not obvious from the code.

14.2 Document index

DOCUMENT	CONTENTS
<code>README.md</code>	Orientation and the short quick start
<code>docs/SETUP_GUIDE.md</code>	Fresh-machine install in detail

DOCUMENT	CONTENTS
<code>docs/RUNBOOK.md</code>	Operations
<code>docs/ARCHITECTURE.md</code>	Module contracts and interfaces
<code>docs/SWAP_THE_LLM.md</code>	The LLM swap in depth
<code>docs/PROMPT_GUIDE.md</code>	Writing and versioning prompts
<code>docs/KNOWN_ISSUES.md</code>	Live issue list
<code>docs/PUBLIC_ACCESS.md</code>	ngrok and public hosting
<code>docs/09_FINAL_DECISIONS.md</code>	The full decision register, security control matrix, deliverability contract
<code>docs/04_LLM_HOSTING.md</code>	Groq setup, model choice, rate limits, the handover swap
<code>docs/adr/</code>	Four ADRs covering the reversed decisions
<code>CLAUDE.md</code>	Project memory — decisions, milestones, measured results

Status at handover

M0–M4, M6–M10 complete and verified. M5 is code-complete and awaiting a visual check in a real email client — the one task that needs a human with Outlook and Gmail.

1,168 tests · 93% coverage · six gates green on Windows and Ubuntu, Python 3.11 and 3.12.

The manual pipeline has generated and delivered real newsletters through Gmail. The automated pipeline has run end to end — discovery, generation, approval by email, scheduled send.

Every dependency is permissively licensed. No model, weights, or inference runtime has ever run on the development machine, and a CI gate makes sure it stays that way.